



НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ  
УНИВЕРСИТЕТ

# Системный дизайн современных приложений

Занятие №1.

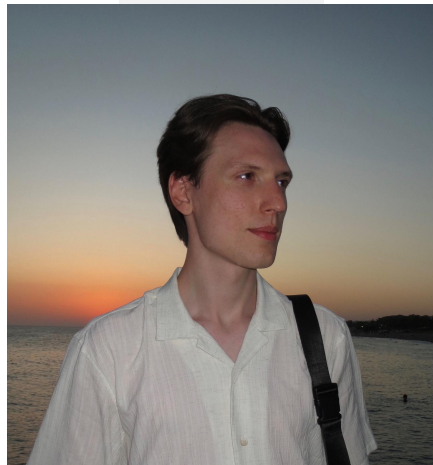
Введение. Основы проектирования систем.



# О курсе [0]

## Преподаватель курса

1. Два года руковожу прекрасными командами и проектами в Холдинге Т1
2. За плечами более 5 лет в финтехе, промышленности, ритейле
3. Бакалавриат и магистратура НИУ ВШЭ с красным дипломом (специальность «Высоконагруженные системы и оптимизация кода»)
4. Начал руководить разработкой в 22 года
5. Преподаю курс «Системный дизайн» в НИУ ВШЭ
6. Неравнодушен к любой программной инженерии



**Николай Савельев**  
**@nikolaysavelev**



# О курсе [1]

## Цель курса

Сформировать навыки проектирования отказоустойчивых распределенных систем, способных выдерживать высокие нагрузки.

USP: через призму реальных ограничений, архитектурных компромиссов и практического применения теоретических концепций.

## Ключевые слова

1. практика
2. архитектура
3. системный дизайн
4. требования к ПО
5. высоконагруженные системы

## Чем курс отличается от других?

1. Фокус на российский IT-контекст
2. Не только hard-skills, но и soft-skills
3. Полезно для всех, кто соприкасается с разработкой
4. Рассматриваем не идеальные варианты, а реальные
5. Полный цикл – от идеи до MVP
6. Закрепляем теорию практикой, разбираемся в технологиях



# О курсе [2]

---

## Формула

**$0.1 * \text{Занятия} + 0.6 * \text{ДЗ} + 0.15 * \text{Тесты} + 0.15 * \text{Защита SDD} + 0.1 * \text{Extra Point (1)}$**

### 1. Работа на занятиях (0.1)

За активную работу 10 баллов. За присутствие 8 баллов.

### 2. ДЗ (0.6)

Будет 6 ДЗ (каждое 0.1).

Домашние задания делятся на 2 категории: system design (3) / practice (3)

- SD - делаем проект (квадратики, кружки).
- Practice - нужно что-то развернуть локально, сделать работу, описать свои мысли в отчете.

### 3. Теоретические тесты (0.15)

Нужно будет ответить на практико-теоретические вопросы. Тесты в середине и конце курса.

### 4. Защита System Design Document (0.15)

Диалог с преподавателем в конце курса.

### 5. Особая признательность и актив (0.1)

Ставится наиболее активным студентам.



# О курсе [3]

## Книги

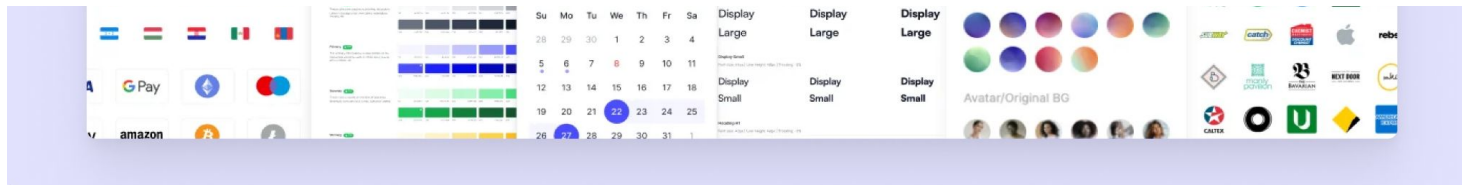
1. М. Клеппман «Высоконагруженные приложения. Программирование, масштабирование, поддержка» (Кабан) - *Гамлет от мира информационных технологий*
2. Б. Бейер «Site Reliability Engineering. Надежность и безотказность как в Google» - *про инженерные подходы для обеспечения работы огромной машины ПО*
3. М. Ричардс «Фундаментальный подход к программной архитектуре: паттерны, свойства, проверенные методы» - *про архитектуру, паттерны, работу архитектором*
4. А. Суй «System Design. Подготовка к сложному интервью» - *про подготовку к секции System Design*
5. Э. Таненбаум «Computer Networks» - *про компьютерные сети, зачем они нужны, как связывают компоненты*
6. Э. Таненбаум «Distributed Systems» - *фундамент с ответами на вопрос «почему распределенные системы победили»*

## Roadmap's

1. <https://github.com/mohsenshafiei/system-design-master-plan>
2. <https://roadmap.sh/system-design>
3. <https://github.com/vladimir-maslov/system-design-roadmap>
4. <https://github.com/donnemartin/system-design-primer> - сборник по СД
5. <https://roadmap.sh/vibe-coding>

## Инструменты

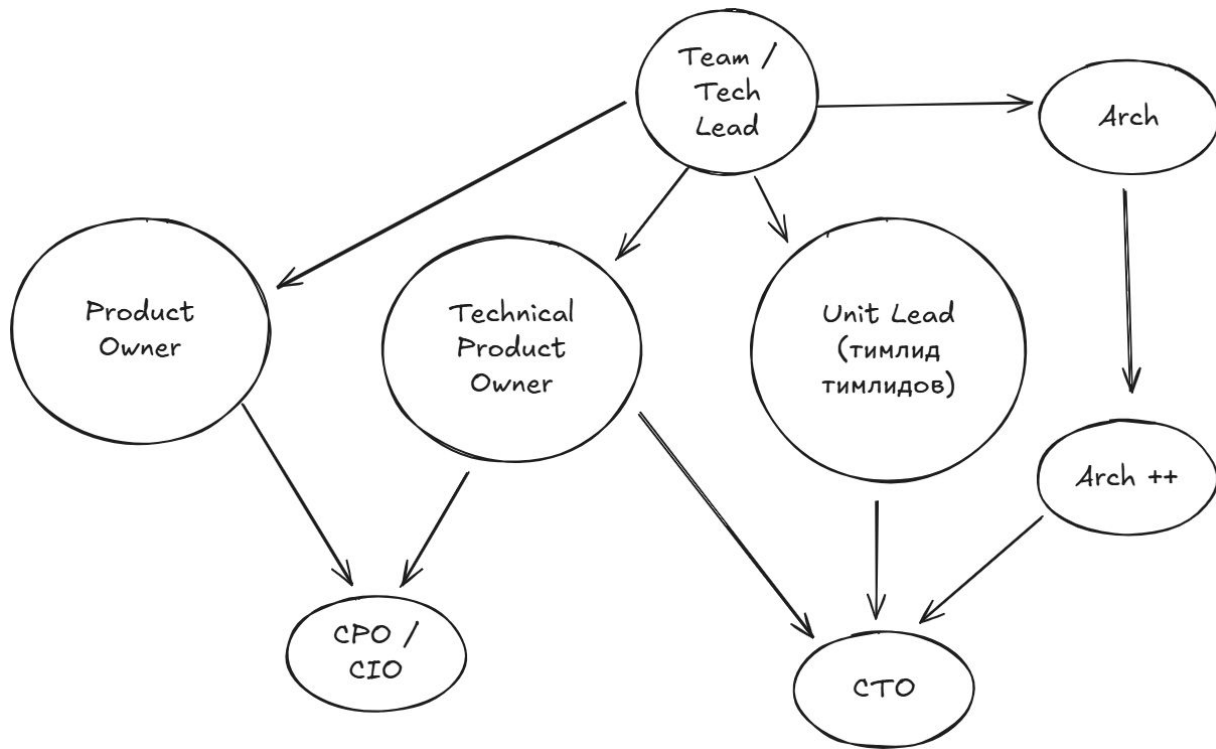
1. Призываю использовать LLM в обучении как советника
2. <https://excalidraw.com/> - для концепт-драфтов
3. <https://plantuml.com/ru/stdlib> - diagram as a code
4. <https://app.diagrams.net/> - для разработки концептов
5. <https://www.architecturalkatas.com/> - Katas
6. <https://systems.education/> - кладезь знаний по проектированию систем
7. <https://system-design.space/> - невероятная кладезь знаний





# Важность конкретно для вас [0]

Пути развития  
инженера





# Важность конкретно для вас [1]

Актуальность на  
рынке

высоконагруженная система

470 вакансий «высоконагруженная система»

По соответствию

За всё время

## Специализации

|                                     |                                           |       |
|-------------------------------------|-------------------------------------------|-------|
| <input checked="" type="checkbox"/> | Программист, разработчик                  | 1 930 |
| <input checked="" type="checkbox"/> | Системный аналитик                        | 482   |
| <input checked="" type="checkbox"/> | Архитектор                                | 471   |
| <input type="checkbox"/>            | Специалист по информационной безопасности | 342   |
| <input type="checkbox"/>            | Другое                                    | 393   |

[Выбрать ещё](#)

## Чем предстоит заниматься:

- участие во встречах с заказчиком, анализ и структурирование потребностей заказчика
- технический пресейл, формирование ТКП, подготовка и проведения презентаций технических решений по ИТ-инфраструктуре (серверы, СХД, СРК, платформы виртуализации, VDI, ЦОД и т.д.)
- разработка ИТ-архитектуры технических решений для решения задач заказчика (развертывание платформы для новых систем, реализация отказоустойчивости и катастрофоустойчивости, модернизация ЦОД и т.д.)





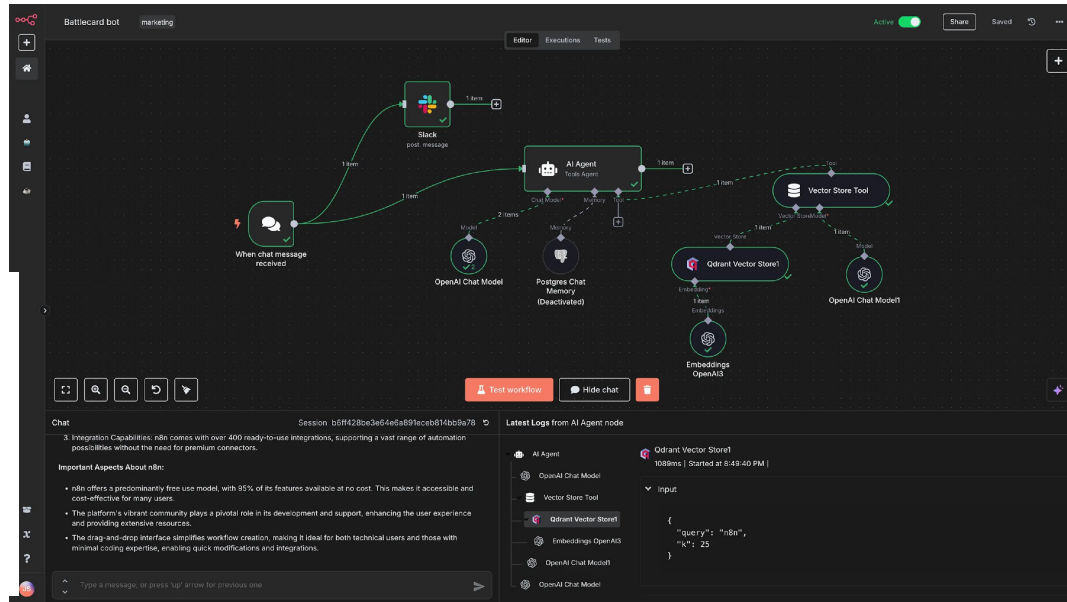
# Важность конкретно для вас [2]

Актуальность в  
ближайшем будущем



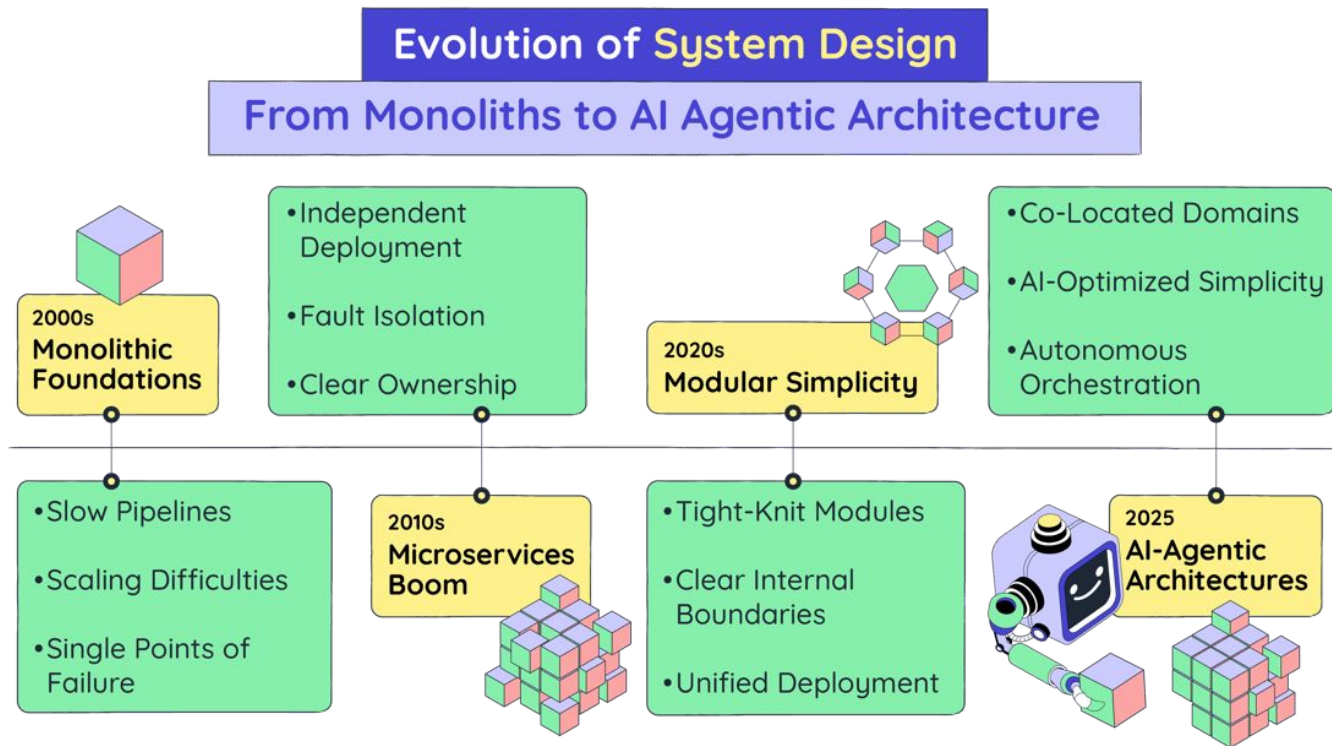
**I Am Developer** ✓  
@iamdeveloper

vibe coding, where 2 engineers can  
now create the tech debt of at least  
50 engineers



# Важность конкретно для вас [2]

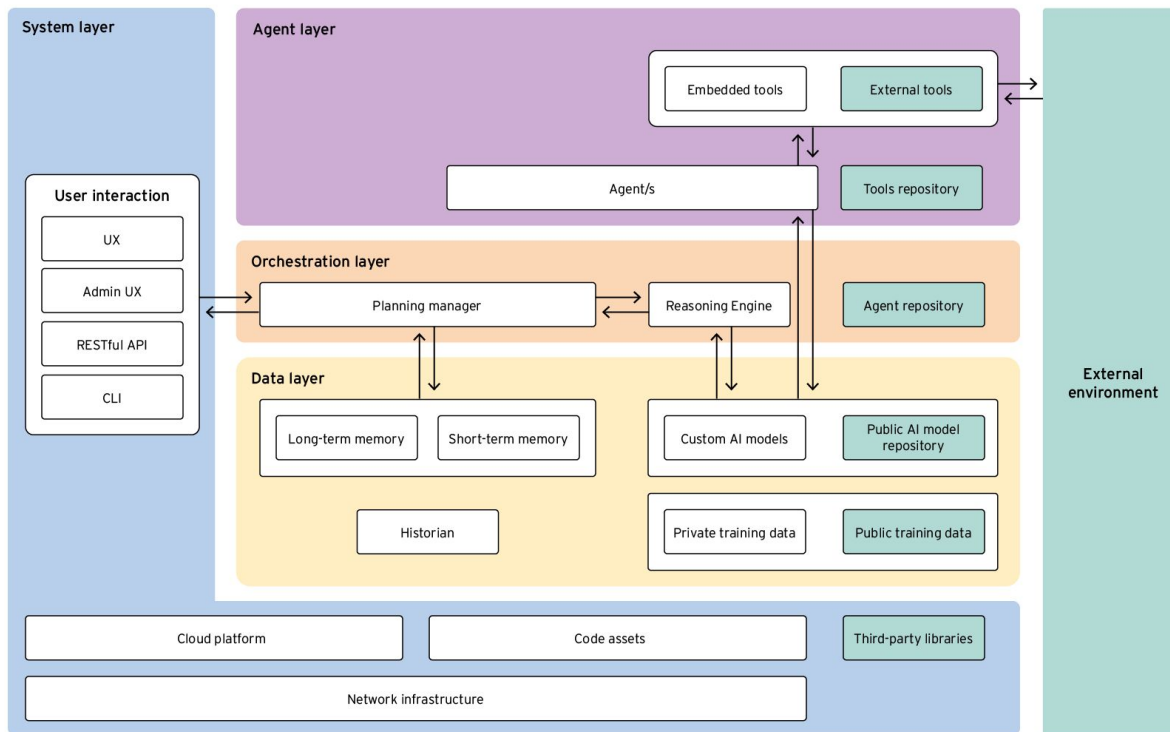
Актуальность в  
ближайшем будущем





# Важность конкретно для вас [2]

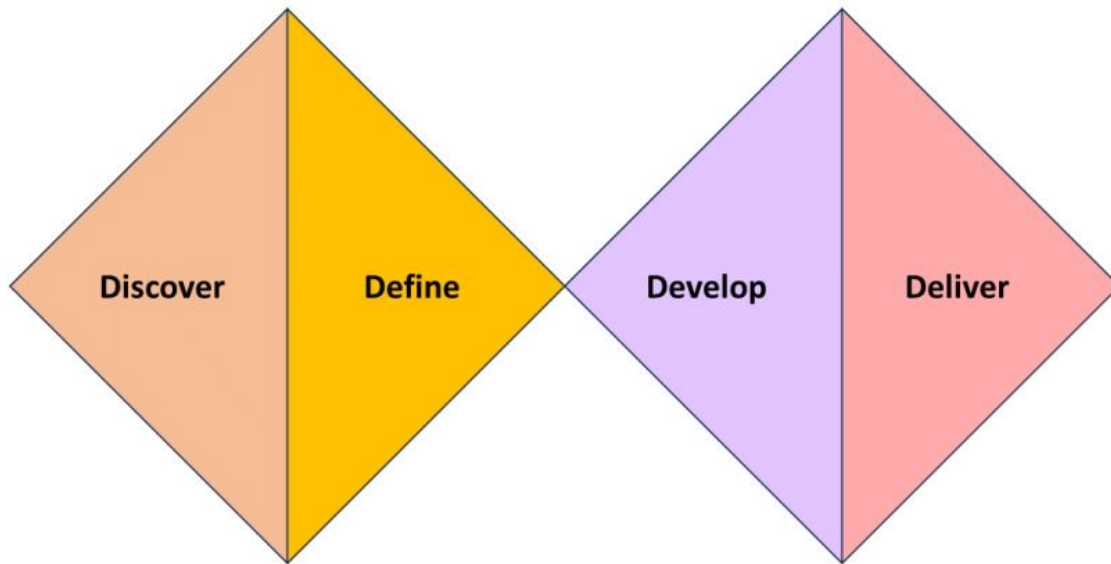
Актуальность в  
ближайшем будущем



©2025 TREND MICRO

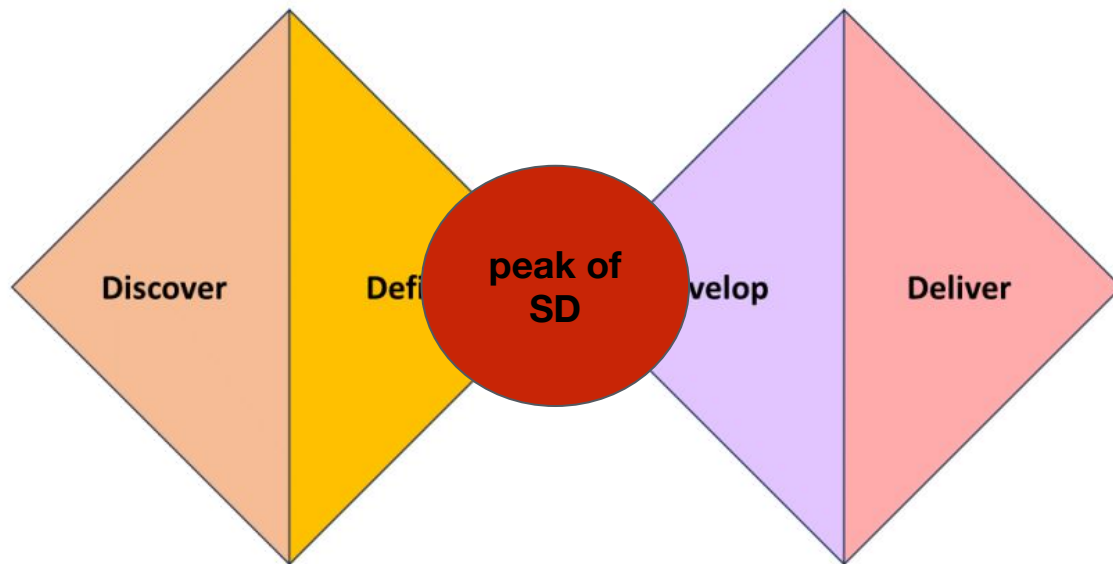
# Важность в целом [0]

---

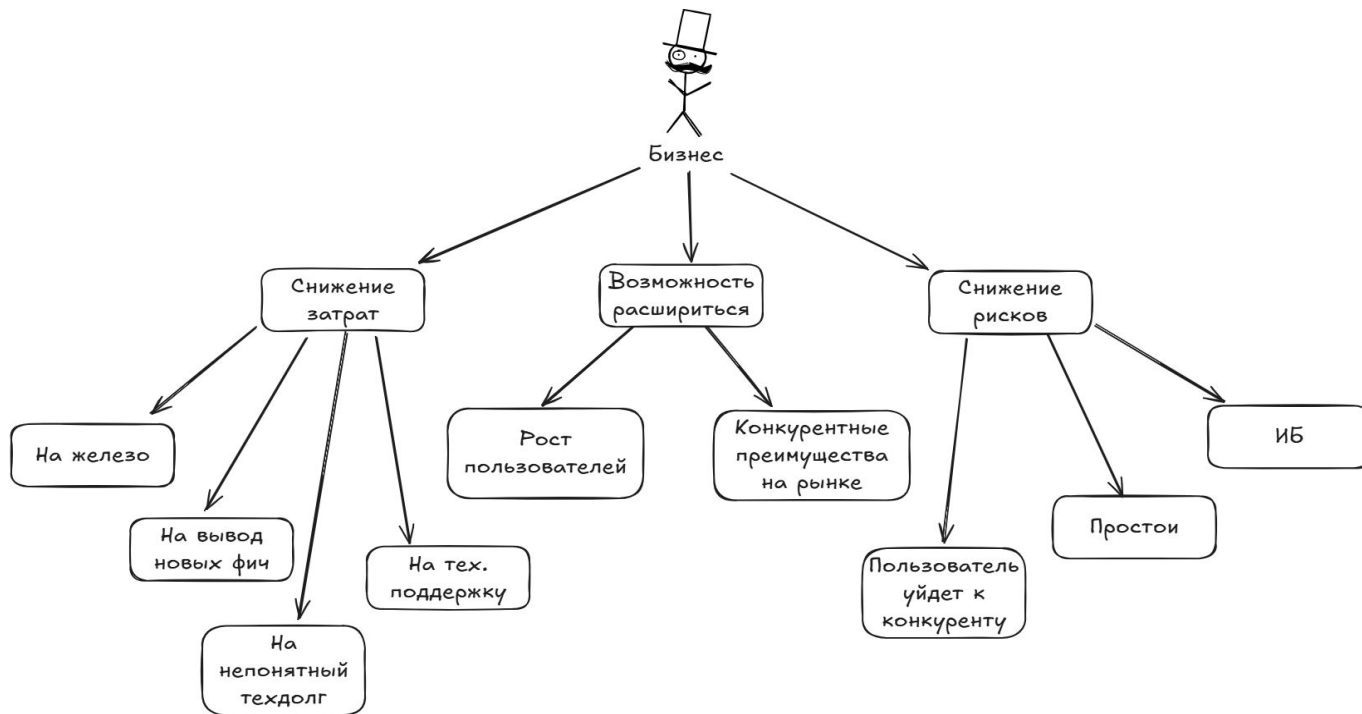


# Важность в целом [0]

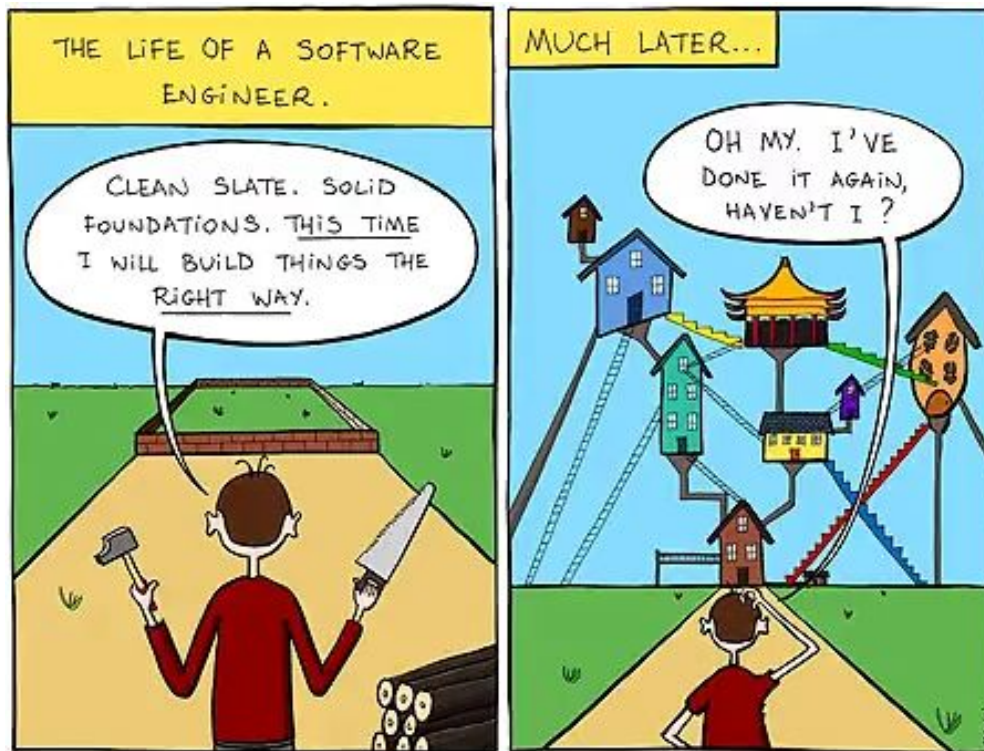
---



# Важность в целом [1]

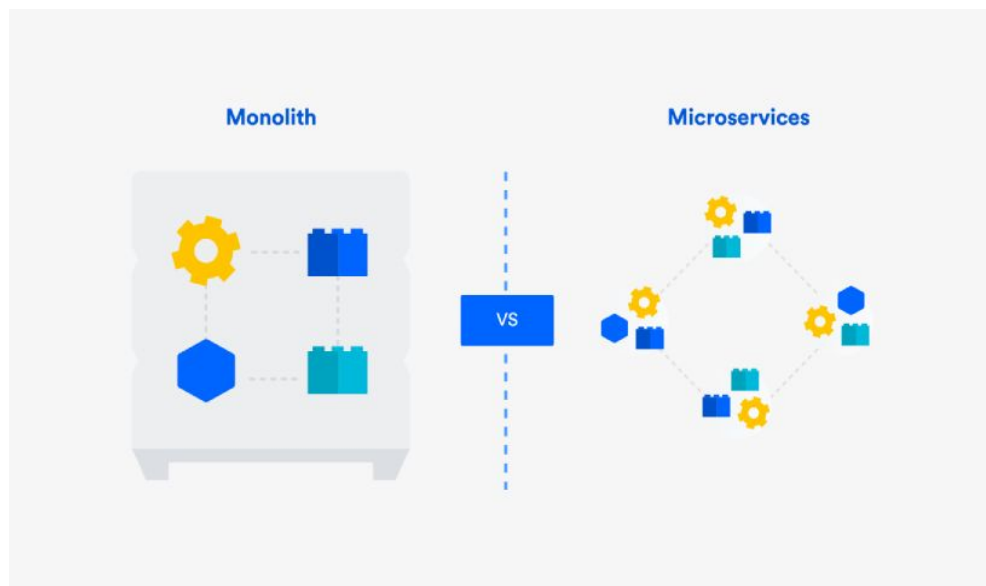


# Стратегия и тактика



## Архитектор vs CFO

Миграция с монолитной архитектуры на микросервисную архитектуру



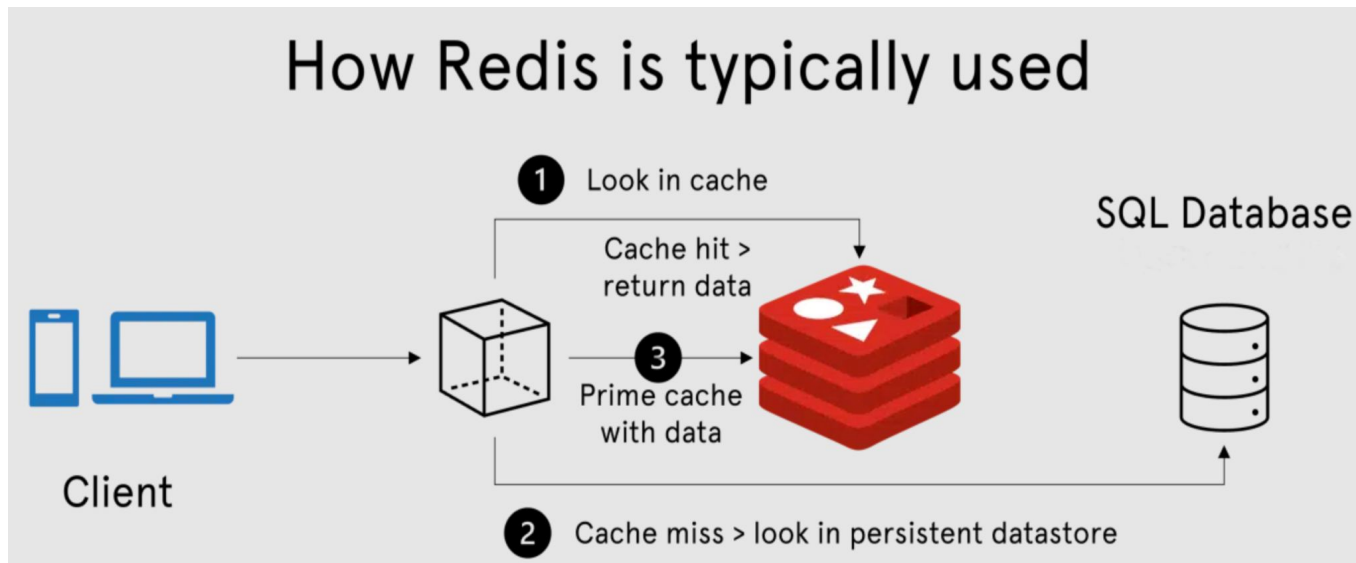




# Убеди стейкхолдера [1]

## IT Lead vs Product Owner

Добавление слоя кэширования к медленной базе данных





# Критерии и итоги

## **Разобраться в мотивации**

Понять, что движет стейкхолдером

## **Общаться на языке собеседника**

Отсутствие специфического жаргона, умение объяснять образами и абстракциями

## **Измеримые значения**

Оперировать конкретными метриками, прокладывать дорогу от техники к деньгам

## **Поиск компромисса**

Предложение итеративного подхода, стремление к win-win

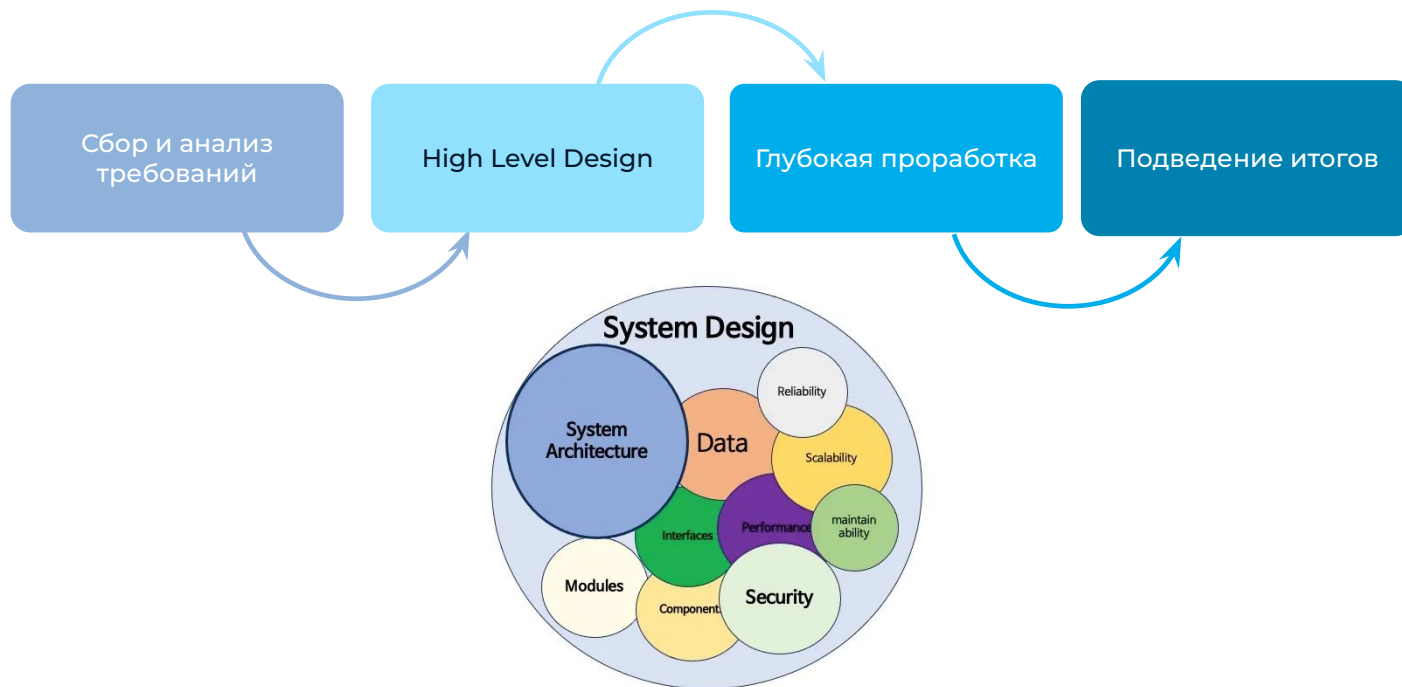
## **Снизить личный интерес**

Не гнаться за технологией ради строчки в резюме

## **Оценка риска**

Быть прозрачным, подсвечивать возможные риски

# Этапность





# Сбор и анализ требований

## Цель этапа

Понять суть задачи: сформировать **полное, непротиворечивое, проверяемое** понимание, что должна делать система.

**Требование** - это представление потребности. Поэтому важно вывести Заказчика на чистую воду, уточнить масштабы решения.

## Важные точки

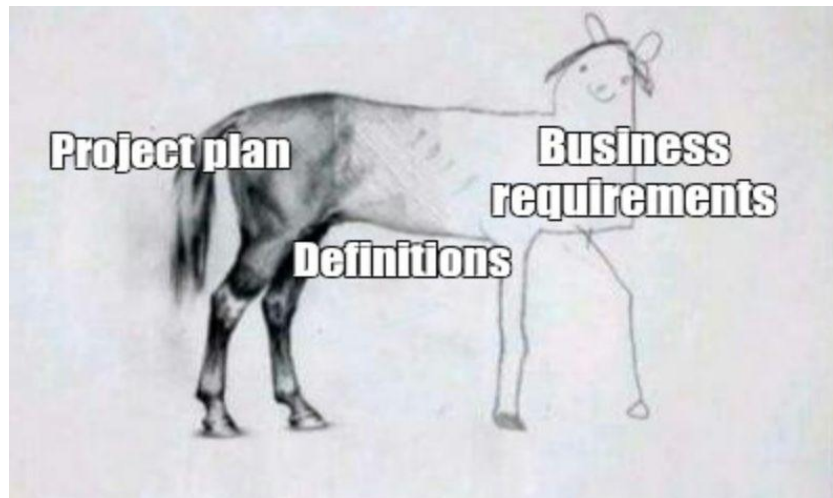
1. Отсев
2. Scope Refinement
3. Функциональные требования
4. Нефункциональные требования
5. Ограничения
6. Расчет нагрузки

## Итоги этапа

Формализованы требования, которые подходят под критерии: **полнота, непротиворечивость, однозначность, проверяемость, трассируемость, реализуемость**

## Что это?

Фаза фильтрации и валидации требований



## Цель

Отделить реализуемые и ценные требования от хотелок, проверить уверенность стейкхолдеров и оценить **готовность проекта к переходу в разработку**.

Любимое:

- Сделаем как у Google, но за 2 недели
- Давайте внедрим ИИ, который сразу все поймет
- Ну это же просто кнопка (под капотом интеграция в 10 систем)

Этап успешен:

- Бизнес транслирует конкретные Acceptance Criteria
- Составлена матрица рисков по требованиям
- Прозрачны ресурсы и почва под задачи



# Scope Refinement

## Что это?

Фаза уточнения и детализации границ проекта

Scope refinement:

1. 3 типа пользователей: клиент, продавец, администратор
2. Выбор вида животного (индивидуальный подбор еды), возможность заказа на любом виде транспорта (включая квадрокоптер), возможность заказать специального кормителя животных, возможность составить свое блюдо на сайте, еда на каждый день (если у вас есть животное, которое участвует в спортивных соревнованиях, то есть возможность подготовить его к ним, доставляя сбалансированное питание).
3. Необязательно: по премиум подписке с вами в зуме созванивается животный диетолог, который проводит скрининг и дает советы. Анализ состояния животного.

## Цель

Сделать границы проекта измеримыми, выявить скрытые зависимости, первый шаг к управлению ожиданиями, подготовка почвы к оценке.

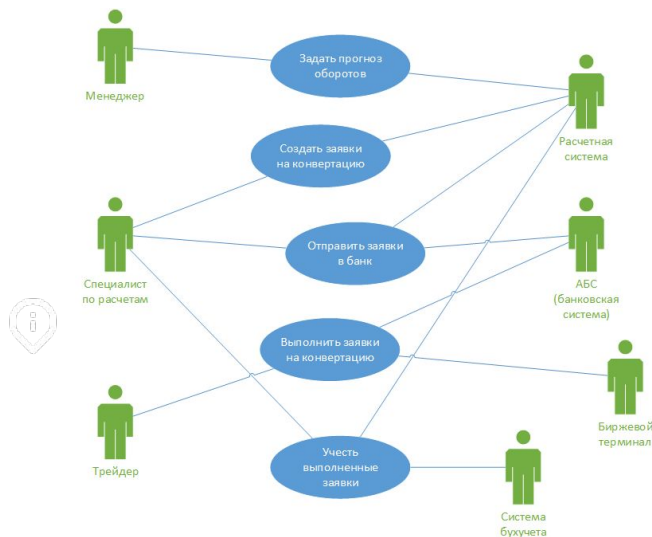
Этап успешен:

- Бизнес получает предсказуемые сроки и бюджет
- Команда получает ясность и уверенность в оценках
- Продукт сфокусирован на важных фичах

# Функциональные требования

## Что это?

Сбор требований, которые отвечают на вопрос «что» система должна делать



## Цель

Собрать и формализовать описание конкретных функций, поведения и реакций системы на входные данные, которые позволяют пользователям достигать своих целей.

### Методики:

- Выделяем акторов, объекты, прикидываем CRUD, дополняем функциями
- Моделирование сценариев на базе, например, CJM: Use Case, User Story, JTBD

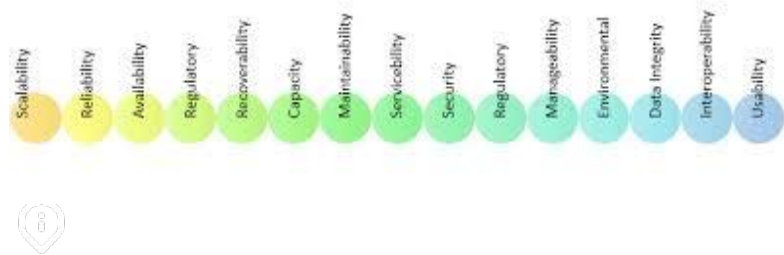
### Ключевые точки:

- Приоритезация функций
- Обработка корнер-кейсов, ошибок
- Трассировка на бизнес-требования

# Нефункциональные требования

## Что это?

Сбор требований, которые отвечают на вопрос «как» система должна работать



## Цель

Прояснить критерии качества и атрибуты системы, которые определяют, как система выполняет свои функции:

- Производительность
- Масштабируемость
- Отказоустойчивость
- Надежность
- Безопасность
- Наблюдаемость
- Регуляторные требования
- Переносимость и совместимость системы
- Пользовательский опыт
- Совместимость





# Q&A

---

1. Что важнее – функциональные или нефункциональные требования?

1. Что важнее – функциональные или нефункциональные требования? *и те, и те. Без ФТ система бесполезна, без НФТ система непригодна*
2. Какое влияние оказывают друг на друга?

1. Что важнее – функциональные или нефункциональные требования? *и те, и те. Без ФТ система бесполезна, без НФТ система непригодна*
2. Какое влияние оказывают друг на друга? *Компромиссная зависимость. Изменение одних может влиять на другие*
3. В чем разница сбора ФТ и НФТ?

| Аспект         | Сбор ФТ                                                           | Сбор НФТ                                                                                     |
|----------------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Источники      | Пользователи, бизнес, эксперты                                    | Чаще всего Tech People: архитекторы, Devops, безопасность, но могут быть и, например, юристы |
| Методы         | Интервью, user stories, use cases                                 | Анализ стандартов, бенчмарки, технические ограничения                                        |
| Вопросы        | "Что должна делать система?"                                      | "Как быстро/надежно/безопасно?"                                                              |
| Формулировки   | Я как пользователь хочу заполнить договор и отправить на проверку | "Время отклика менее 2 сек при 1000 пользователей"                                           |
| Когда выявляют | При общении с бизнесом, как правило в начале                      | Часто упускают, нужно спрашивать явно                                                        |



# Ограничения

1. Бизнес-модель: B2B / B2C / B2G
2. Бизнес-метрики и масштабирование: DAU, RETENTION
3. География пользователей: город, страна, мир
4. Специфика домена: екоммерс, финтех, кибербез
5. Тип устройств: мобилка, десктоп, браузер
6. Этап развития: MVP / Full-Scale Product
7. Тип развертывания: Cloud / On-Premise
8. Время разработки
9. Бюджет на технологии
10. Технологии: ТК компании, переиспользование
11. Существующие сервисы
12. Бюджет на железо
13. SLA: доступность, производительность
14. Регуляторные: юридические нормы
15. Интеграции
16. Санкции и импортозамещение



# Расчет нагрузки

## Что это?

Хотим прогнозировать сколько денег потратим на инфраструктуру и какие компоненты нам нужны.

- Пользовательский трафик, сценарии использования, RPS
- Сетевой трафик и соединения
- Нагрузка на вычислительную мощность
- Необходимое дисковое пространство для хранения данных и расчет потенциального прироста объема хранилища

### Трафик

- + В сутках 86 400 секунд.
- + DAU обычно 10-30% от MAU.
- + Пиковый час: 10-20% от всего суточного трафика.
- + Black Friday, праздники, рабочие дни.
- +  $RPS = (DAU * \text{Среднее кол-во действий в день} * \text{Пиковый коэффициент}) / 86\,400$ .
- + Пиковый коэффициент - берем из логических рассуждений. От 1 до 10

### Данные

- + Средний размер HTTP-запроса с форматом JSON: 1 KB (= 1024 B).
- + Изображение: размер сильно отличается. Но в среднем 10-1000 KB.
- + Видео (можно записать себе сколько секунда видео в 720p и 1080p): 1-10 MB.
- + Не забываем про реплики и бэкапы.

### Базы данных

- + Отношение чтение/запись:
  - + Для систем с множеством контента: 80/20
  - + Для стандартных систем: 60/40
  - + IoT: 30/70
- + Лимиты БД:
  - + SQL: ~500-1000 QPS на инстанс
  - + NoSQL: 10k-100k QPS на инстанс

### Полезные приближения

- + В 1 GB: ~1 миллион JSON-объектов
- + В 1 TB: ~250 миллионов страниц текста
- + 1 сервер (8 ядер): 5k-50k RPS в зависимости от сложности запросов



# Домашнее задание

## Задача

1. Нажать “Do one!” на этом сайте: <https://www.architecturalkatas.com>
2. Ознакомиться с вводными
3. Целиком пройти этап “Сбор и анализ требований”
  1. Отсев
  2. Scope Refinement
  3. Функциональные требования
  4. Нефункциональные требования
  5. Ограничения
  6. Расчет нагрузки
4. Сформировать отчет по этим пунктам в любом читаемом формате
5. Доп. вводные: вы находитесь в России, 2026 год



НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ  
УНИВЕРСИТЕТ